

claude-windows / 42211f80-2169-4025-ad9d-99c1aa29ff70

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\42211f80-2169-4025-ad9d-99c1aa29ff70\subagents\workflows\wf_d4359b2a-456\agent-a6d265a01cb9e02cc.jsonl`  
- SHA-256: `9b4432a69ed8e98e1beb27e80bfe0da065b175cf80673c957b5b93bbb06d73a9`  
- Source modified: `2026-06-06T15:22:41+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_d4359b2a-456`  
- session_id: `42211f80-2169-4025-ad9d-99c1aa29ff70`
```

Transcript

1. user (2026-06-06T15:20:37.514Z)

You are mapping ONE subsystem of the "badminton-highlight-indexer" Python repo (cwd is the repo root) for a dense CODE MAP that a future AI engineer will read to ramp up FAST on the codebase without crawling files.

Subsystem: Orchestration & config

Files: run_process_and_stitch.py, run_eval.py, run_phase3_eval.py, run_regression.py, backend/main.py, config.json, setup.py

READ each listed file (use Read; use Grep/Glob to resolve any directory globs to actual files). Then return, per file:

- path (exact, repo-relative, forward slashes)
- role: one terse line ? what the file is for
- symbols: the IMPORTANT classes / functions / dataclasses / module-level constants & registries. For each: name, kind (class|func|dataclass|const|registry), a terse signature (params + return where useful), and a one-line role. SKIP trivial private helpers and boilerplate.
- contracts: exact data shapes this file defines or consumes (dict keys, CSV column orders, tuple layouts, dataclass fields, return types). Be precise ? these are the cross-module interfaces.
- edges: calls/imports to OTHER files that matter for dataflow, each as "thisSymbol -> otherFile.thing (why)".
- gotchas: non-obvious constraints, footguns, platform/WSL/GPU/threadpool specifics, dead paths, defaults that bite.

Rules: be terse and ACCURATE. Every symbol and path MUST actually exist in the file ? do NOT invent or guess. If something is notable but you're unsure, omit it. Return via the structured schema.

2. assistant (2026-06-06T15:20:45.766Z)

I'll map the Orchestration & config subsystem. Let me read all the listed files.

3. user (2026-06-06T15:20:46.779Z)

```
1      import os  
2      import sys  
3      import json  
4      import hashlib  
5      import logging  
6      from dotenv import load_dotenv  
7  
8      load_dotenv()  
9  
10     # Configure root logger ? all backend modules use logging.getLogger(__name__)
```

```

11     logging.basicConfig(
12         level=logging.INFO,
13         format="%asctime)s [%(name)s] %(levelname)s: %(message)s",
14         datefmt="%H:%M:%S",
15     )
16
17     from backend.database import Database
18     from backend.validators.base import registry
19     from backend.indexer import segmenter_registry
20     from backend.stitcher.compiler import VideoCompiler
21
22     def get_file_md5(filepath: str) -> str:
23         total_size = os.path.getsize(filepath)
24         hasher = hashlib.md5()
25         processed = 0
26         with open(filepath, "rb") as f:
27             # Read in 1MB chunks for faster IO on huge files
28             for chunk in iter(lambda: f.read(1024 * 1024), b''):
29                 hasher.update(chunk)
30                 processed += len(chunk)
31                 if processed % (1024 * 1024 * 50) == 0 or processed == total_size:
32                     print(f"\r[MD5] Processed {processed/(1024**3):.2f} GB /
{total_size/(1024**3):.2f} GB ({(processed/total_size)*100:.1f}%)", end="", flush=True)
33             print()
34             return hasher.hexdigest()
35
36     def main():
37         video_path = r"C:\Users\avidu\Videos\Testing\KushagraYashAviFirstVideo.mp4"
38         output_dir = r"C:\Users\avidu\Videos\Testing"
39         output_filename = "Final_2_OutputOfKushagraYashAviFirstVideo.mp4"
40
41         print(f>Loading config...)
42         if os.path.exists("config.json"):
43             with open("config.json", "r") as f:
44                 config = json.load(f)
45         else:
46             config = {}
47
48         print(f>Verifying files exist...)
49         if not os.path.exists(video_path):
50             print(f>Error: Video file not found at {video_path}")
51             return
52
53         os.makedirs("output", exist_ok=True)
54         db_path = os.path.join("output", "sports_indexer.db")
55         db = Database(db_path)
56
57         # Calculate MD5 of the video to uniquely identify it
58         import time
59         print("Calculating video MD5 (this might take a few seconds on large files)...",
flush=True)
60         t0 = time.time()
61         video_id = get_file_md5(video_path)
62         print(f"Calculated MD5: {video_id} (took {time.time()-t0:.1f}s)", flush=True)
63
64         stitching_config = config.get("stitching", {})
65         use_cache = stitching_config.get("use_cache", True)
66
67         segments = []
68         cached_video = db.get_video(video_id)
69         cache_hit = False
70
71         if cached_video and cached_video.get("status") == "processed":
72             segments = db.query_play_segments(video_id, {})
73             if len(segments) > 0:

```

```

74         print(f"\n[CACHE] Found cached indexed video in database (MD5: {video_id})
with {len(segments)} parsed play segments.", flush=True)
75         choice = input("Would you like to (1) Run stitching on the cached segments,
or (2) Reprocess the video with Gemini from scratch? [1/2]: ").strip()
76         if choice == '1':
77             print("[CACHE] Using cached play segments. Skipping segmenter API
call.", flush=True)
78             cache_hit = True
79         else:
80             print("[CACHE] Ignoring cache...", flush=True)
81         else:
82             print(f"[CACHE] Video exists in DB but contains 0 play segments. Treating as
cache miss.", flush=True)
83
84         if not cache_hit:
85             print("[CACHE] Initializing clean indexing...", flush=True)
86             db.clear_video_data(video_id)
87
88             print("Bypassing OpenCV validations for performance on massive files...",
flush=True)
89             db.add_video(video_id, video_path, "processed", [])
90
91             print("Running segmenter & indexing...", flush=True)
92             seg_name = config.get("indexing", {}).get("default_segmenter", "yolo_hybrid")
93             segmenter_cls = segmenter_registry.get(seg_name)
94             if not segmenter_cls:
95                 print(f"[FAIL] Segmenter '{seg_name}' not found.")
96                 return
97             print(f"Using segmenter: {seg_name}", flush=True)
98             segmenter = segmenter_cls(db, config)
99             segments = segmenter.process_video(video_id, video_path)
100            print(f"Successfully indexed {len(segments)} play segments.", flush=True)
101
102            if not segments:
103                print("No play segments detected. Cannot compile highlights.")
104                return
105
106
107            end_padding = stitching_config.get("end_padding", 1.0)
108            begin_padding = stitching_config.get("begin_padding", 0.5)
109            min_shot_count = stitching_config.get("min_shot_count", 0)
110
111            if cache_hit:
112                print("\n--- Stitching Customizations ---")
113                try:
114                    bpad_input = input(f"Enter begin_padding in seconds (default
{begin_padding}): ").strip()
115                    if bpad_input:
116                        begin_padding = float(bpad_input)
117
118                    pad_input = input(f"Enter end_padding in seconds (default {end_padding}):
").strip()
119                    if pad_input:
120                        end_padding = float(pad_input)
121
122                    shot_input = input(f"Enter min_shot_count to filter by (default
{min_shot_count}): ").strip()
123                    if shot_input:
124                        min_shot_count = int(shot_input)
125                except ValueError:
126                    print("[WARNING] Invalid input detected, falling back to defaults from
config.json.")
127
128            print(f"\nCompiling highlights (with begin_padding={begin_padding}s,
end_padding={end_padding}s, min_shot_count={min_shot_count})...", flush=True)

```

```

129
130     # Filter segments where shot_count >= min_shot_count
131     time_pairs = []
132     skipped_count = 0
133     for s in segments:
134         meta = s.get("metadata", {})
135         if isinstance(meta, str):
136             try:
137                 meta = json.loads(meta)
138             except Exception:
139                 meta = {}
140         shot_count = meta.get("shot_count", 0)
141         if shot_count >= min_shot_count:
142             time_pairs.append((s["start_time"], s["end_time"]))
143         else:
144             skipped_count += 1
145
146     if skipped_count > 0:
147         print(f"[FILTER] Excluded {skipped_count} play segments with fewer than
{min_shot_count} shots.", flush=True)
148     print(f"[FILTER] Retaining {len(time_pairs)} play segments for stitching.",
flush=True)
149
150     if not time_pairs:
151         print("No play segments passed the filtering criteria. Cannot compile
highlights.")
152         return
153
154     compiler = VideoCompiler(output_dir, end_padding=end_padding,
begin_padding=begin_padding)
155     try:
156         out_path = compiler.compile_highlights(video_path, time_pairs, output_filename)
157         print(f"Stitched highlight video saved to: {out_path}", flush=True)
158     except Exception as e:
159         print(f"Error during stitching: {e}", flush=True)
160
161     if __name__ == "__main__":
162         main()
163

```

4. user (2026-06-06T15:20:47.658Z)

```

1     """CLI to evaluate detected rallies against hand-annotated ground truth.
2
3     Scores predictions already stored in the SQLite DB (run the pipeline first) ? no
4     API calls, fully deterministic. See docs/EVALUATION.md for the workflow.
5
6     Examples
7     -----
8     Create an empty annotation file to fill in:
9         python run_eval.py --scaffold --annotations rallies.csv
10
11     Score a video's detected rallies (identify it by file or by stored video_id):
12         python run_eval.py --video "C:/path/match.mp4" --annotations rallies.csv
13         python run_eval.py --video-id <md5> --annotations rallies.csv --stage both --iou 0.5
14
15     Show which rallies were missed / spurious, and dump a JSON report:
16         python run_eval.py --video-id <md5> --annotations rallies.csv --show-failures --json
report.json
17     """
18
19     import os
20     import sys
21     import json
22     import hashlib

```

```

23     import argparse
24     import logging
25
26     from backend.database import Database
27     from backend.eval.annotations import load_annotations, write_scaffold
28     from backend.eval.harness import evaluate, format_report_text, report_to_dict, STAGES
29
30
31     def get_file_md5(filepath: str) -> str:
32         """MD5 of a file, chunked ? matches how the pipeline derives video_id."""
33         hasher = hashlib.md5()
34         with open(filepath, "rb") as f:
35             for chunk in iter(lambda: f.read(8192), b""):
36                 hasher.update(chunk)
37         return hasher.hexdigest()
38
39
40     def _default_db_path() -> str:
41         """Resolve the DB path from config.json (output.db_name) under output/."""
42         db_name = "sports_indexer.db"
43         if os.path.exists("config.json"):
44             try:
45                 with open("config.json") as f:
46                     db_name = json.load(f).get("output", {}).get("db_name", db_name)
47             except (json.JSONDecodeError, OSError):
48                 pass
49         return os.path.join("output", db_name)
50
51
52     _EPILOG = """\
53     Prerequisite:
54         The harness scores predictions read from the SQLite DB ? it does NOT run the
55         pipeline. So you must first PROCESS the video (e.g. via run_process_and_stitch.py)
56         so its detections land in output/sports_indexer.db. Then point this tool at the
57         same video file (--video) or its stored MD5 (--video-id). If you get
58         "database not found" or zero predictions, the video hasn't been processed yet.
59
60     See docs/EVALUATION.md for the annotation format and how to read the output.
61     """
62
63
64     def build_parser() -> argparse.ArgumentParser:
65         p = argparse.ArgumentParser(
66             description="Evaluate detected rallies vs ground-truth annotations.",
67             epilog=_EPILOG,
68             formatter_class=argparse.RawDescriptionHelpFormatter,
69         )
70         p.add_argument("--annotations", help="Path to the ground-truth rally CSV (start,end per row).")
71         p.add_argument("--video", help="Path to the video file (its MD5 becomes the video_id).")
72         p.add_argument("--video-id", help="Stored video_id (MD5) to evaluate, instead of --video.")
73         p.add_argument("--db", default=None, help="SQLite DB path (default: from config.json).")
74         p.add_argument("--stage", choices=("candidates", "final", "both"), default="both",
75             help="Which prediction stage(s) to score (default: both).")
76         p.add_argument("--detector", default=None,
77             help="Only score candidates from this detector (e.g. yolo_hybrid, tracknet_hybrid).")
78         p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default: 0.5).")
79         p.add_argument("--pr-curve", action="store_true", help="Include an IoU sweep (0.1..0.9) in the report.")
80         p.add_argument("--show-failures", action="store_true",

```

```

81             help="List missed (FN) and spurious (FP) rallies with timestamps.")
82     p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report to
this path.")
83     p.add_argument("--scaffold", action="store_true",
84                     help="Write an empty annotation CSV to --annotations and exit.")
85     return p
86
87
88     def main(argv=None) -> int:
89         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
90         args = build_parser().parse_args(argv)
91
92         # Scaffold mode: just write the template and exit.
93         if args.scaffold:
94             if not args.annotations:
95                 print("ERROR: --scaffold requires --annotations <path>", file=sys.stderr)
96                 return 2
97             write_scaffold(args.annotations)
98             print(f"Wrote empty annotation file: {args.annotations}")
99             return 0
100
101         if not args.annotations:
102             print("ERROR: --annotations is required (or use --scaffold).", file=sys.stderr)
103             return 2
104
105         # Resolve the video_id.
106         if args.video_id:
107             video_id = args.video_id
108         elif args.video:
109             if not os.path.exists(args.video):
110                 print(f"ERROR: video not found: {args.video}", file=sys.stderr)
111                 return 2
112             print("Computing video MD5...")
113             video_id = get_file_md5(args.video)
114             print(f"video_id = {video_id}")
115         else:
116             print("ERROR: provide --video <path> or --video-id <md5>.", file=sys.stderr)
117             return 2
118
119         db_path = args.db or _default_db_path()
120         if not os.path.exists(db_path):
121             print(f"ERROR: database not found at {db_path}. Run the pipeline first.",
file=sys.stderr)
122             return 2
123
124         gt = load_annotations(args.annotations)
125         if not gt:
126             print(f"WARNING: no ground-truth rallies in {args.annotations}.",
file=sys.stderr)
127
128         stages = list(STAGES) if args.stage == "both" else [args.stage]
129         db = Database(db_path)
130         report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
131                         detector=args.detector, with_pr_curve=args.pr_curve)
132
133         print(format_report_text(report, show_failures=args.show_failures))
134
135         if args.json_out:
136             with open(args.json_out, "w") as f:
137                 json.dump(report_to_dict(report), f, indent=2)
138                 print(f"\nJSON report written to {args.json_out}")
139
140         return 0
141
142

```

```
143     if __name__ == "__main__":
144         raise SystemExit(main())
145
```

5. user (2026-06-06T15:20:48.128Z)

```
1     """Phase 3 driver: batch process + evaluate a collector export against ground truth.
2
3     Walks the `manifest.json` produced by sports-data-collector's `curate export`,
4     processes each video through a segmenter (so detections land in the indexer DB),
5     scores each against its ground-truth rally CSV, and prints per-video plus
6     corpus-aggregate precision/recall/F1, mean IoU, and boundary error.
7
8     Examples
9     -----
10    Free CPU baseline on a single video (validate the loop):
11        python run_phase3_eval.py --manifest ../sports-data-
12        collector/data/eval_export/manifest.json \
13        --segmenter motion --only shuttlestet_1
14
15    Full corpus with the two-stage detector (needs GEMINI_API_KEY; costs API calls):
16        python run_phase3_eval.py --manifest ../manifest.json --segmenter yolo_hybrid \
17        --json output/phase3_report.json
18
19    Re-score only (videos already processed), no re-segmentation:
20        python run_phase3_eval.py --manifest ../manifest.json --score-only
21    """
22
23    import os
24    import sys
25    import json
26    import time
27    import hashlib
28    import argparse
29    import logging
30
31    from dotenv import load_dotenv
32    load_dotenv() # make GEMINI_API_KEY (etc.) available to the AI-backed segmenters
33
34    from backend.database import Database
35    # Importing the package registers all segmenters (motion/gemini/yolo_hybrid/...).
36    from backend.indexer import segmenter_registry
37    from backend.eval.annotations import load_annotations
38    from backend.eval.harness import evaluate, format_report_text, report_to_dict
39    from backend.eval import batch
40
41    logger = logging.getLogger(__name__)
42
43    def get_file_md5(filepath: str) -> str:
44        hasher = hashlib.md5()
45        with open(filepath, "rb") as f:
46            for chunk in iter(lambda: f.read(65536), b""):
47                hasher.update(chunk)
48        return hasher.hexdigest()
49
50
51    def load_config(path: str = "config.json") -> dict:
52        if os.path.exists(path):
53            with open(path) as f:
54                return json.load(f)
55        return {}
56
57
58    def build_parser() -> argparse.ArgumentParser:
```

```

59     p = argparse.ArgumentParser(
60         description="Batch process + evaluate a collector export manifest (Phase 3).",
61         formatter_class=argparse.RawDescriptionHelpFormatter,
62     )
63     p.add_argument("--manifest", required=True, help="Path to the collector's
manifest.json.")
64     p.add_argument("--videos-root", default=None,
65         help="Root for resolving manifest relative local_paths "
66         "(default: the collector repo root, two levels above the
manifest).")
67     p.add_argument("--segmenter", default="motion",
68         help="Segmenter to run (default: motion ? free/CPU). "
69         "yolo_hybrid/gemini need GEMINI_API_KEY and cost API calls.")
70     p.add_argument("--stage", choices=("candidates", "final", "both", "auto"),
default="auto",
71         help="Which prediction stage(s) to score (default: auto by
segmenter).")
72     p.add_argument("--detector", default=None, help="Restrict candidate scoring to this
detector.")
73     p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default:
0.5).")
74     p.add_argument("--only", action="append", default=None,
75         help="Only this video_id (repeatable).")
76     p.add_argument("--limit", type=int, default=None, help="Process at most N videos.")
77     p.add_argument("--max-frames", type=int, default=None,
78         help="Cap segmenter frames (fast validation).")
79     p.add_argument("--score-only", action="store_true",
80         help="Skip segmentation; only score detections already in the DB.")
81     p.add_argument("--reprocess", action="store_true",
82         help="Re-run segmentation even if the video already has segments.")
83     p.add_argument("--db", default=None, help="Indexer DB path (default:
output/<db_name>).")
84     p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report
here.")
85     p.add_argument("--show-failures", action="store_true", help="List missed/spurious
rallies per video.")
86     return p
87
88
89 def main(argv=None) -> int:
90     logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
91     args = build_parser().parse_args(argv)
92
93     manifest_path = os.path.abspath(args.manifest)
94     if not os.path.exists(manifest_path):
95         print(f"ERROR: manifest not found: {manifest_path}", file=sys.stderr)
96         return 2
97     manifest_dir = os.path.dirname(manifest_path)
98     videos_root = args.videos_root or os.path.dirname(os.path.dirname(manifest_dir))
99
100     with open(manifest_path) as f:
101         manifest = json.load(f)
102     entries = manifest.get("eval_sets", [])
103
104     cfg = load_config()
105     db_name = cfg.get("output", {}).get("db_name", "sports_indexer.db")
106     db_path = args.db or os.path.join("output", db_name)
107     os.makedirs(os.path.dirname(db_path) or ".", exist_ok=True)
108     db = Database(db_path)
109
110     stages = batch.default_stages_for(args.segmenter, args.stage)
111
112     # Resolve the segmenter class up front (unless purely scoring).
113     segmenter = None
114     if not args.score_only:

```

```

115         seg_cls = segmenter_registry.get(args.segmenter)
116         if not seg_cls:
117             print(f"ERROR: segmenter '{args.segmenter}' not found. "
118                   f"Available: {list(segmenter_registry.list_available().keys())}",
119                   file=sys.stderr)
120         return 2
121         segmenter = seg_cls(db, cfg)
122
123     # Filter entries.
124     targets = []
125     for e in entries:
126         if args.only and e["video_id"] not in args.only:
127             continue
128         targets.append(e)
129     if args.limit:
130         targets = targets[: args.limit]
131
132     per_video = []
133     skipped = []
134     print(f"Phase 3: {len(targets)} target(s) | segmenter={args.segmenter} |
135           stages={stages} | db={db_path}\n")
136
137     for e in targets:
138         vid = e["video_id"]
139         video_path = batch.resolve_video_path(e.get("local_path"), videos_root)
140         csv_path = os.path.join(manifest_dir, e["csv"])
141
142         if not video_path or not os.path.exists(video_path):
143             skipped.append((vid, "no video file"))
144             print(f"[skip] {vid}: no video file ({e.get('local_path')})")
145             continue
146         if not os.path.exists(csv_path):
147             skipped.append((vid, "no GT csv"))
148             print(f"[skip] {vid}: GT csv missing ({csv_path})")
149             continue
150
151         video_id = get_file_md5(video_path)
152
153         # Segment unless score-only or already present (and not --reprocess).
154         if not args.score_only:
155             already = db.query_play_segments(video_id, {}) or
156             db.query_candidate_segments(video_id)
157             if already and not args.reprocess:
158                 print(f"[have] {vid}: detections already in DB (use --reprocess to
159                       redo)")
160             else:
161                 print(f"[proc] {vid}: segmenting with {args.segmenter}...")
162                 db.add_video(video_id, video_path, "processed", [])
163                 t0 = time.time()
164                 segs = segmenter.process_video(video_id, video_path,
165                 max_frames=args.max_frames)
166                 print(f"          -> {len(segs)} segments in {time.time() - t0:.0f}s")
167
168         gt = load_annotations(csv_path)
169         report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
170                           detector=args.detector)
171         print(format_report_text(report, show_failures=args.show_failures))
172         print()
173         d = report_to_dict(report)
174         d["collector_video_id"] = vid
175         per_video.append(d)
176
177     agg = batch.aggregate(per_video, stages)
178     print(batch.format_aggregate(agg))
179     if skipped:

```

```

174         print(f"\nSkipped {len(skipped)}: " + ", ".join(f"{v} ({why})" for v, why in
skipped))
175
176     if args.json_out:
177         with open(args.json_out, "w") as f:
178             json.dump({"segmenter": args.segmenter, "iou": args.iou, "stages": stages,
179                       "aggregate": agg, "per_video": per_video,
180                       "skipped": skipped}, f, indent=2)
181             print(f"\nJSON report written to {args.json_out}")
182
183     return 0
184
185
186 if __name__ == "__main__":
187     raise SystemExit(main())
188

```

6. user (2026-06-06T15:20:48.625Z)

```

1     """CLI: regression-test the production tool against golden-set ground truth.
2
3     Obtains ground truth from an annotation provider (a tier), scores the video's
4     stored predictions, and compares to a snapshotted baseline. Exit code 1 on a
5     detected regression so it can gate CI. See docs/GOLDEN_SET_IMPLEMENTATION.md.
6
7     Examples
8     -----
9     Snapshot the current numbers as the baseline (first run / after a real improvement):
10     python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
--update-baseline
11
12     Check for a regression (exit 1 if precision/recall dropped beyond tolerance):
13     python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
14     """
15     import os
16     import sys
17     import json
18     import argparse
19     import logging
20
21     from backend.database import Database
22     from backend.annotations import annotation_registry
23     from backend.eval import regression
24
25
26     def _default_db_path() -> str:
27         db_name = "sports_indexer.db"
28         if os.path.exists("config.json"):
29             try:
30                 with open("config.json") as f:
31                     db_name = json.load(f).get("output", {}).get("db_name", db_name)
32             except (json.JSONDecodeError, OSError):
33                 pass
34         return os.path.join("output", db_name)
35
36
37     def build_parser() -> argparse.ArgumentParser:
38         p = argparse.ArgumentParser(description="Regression-test the tool vs golden-set
ground truth.")
39         p.add_argument("--video-id", required=True, help="Stored video_id (MD5) whose
predictions to score.")
40         p.add_argument("--tier", default="file",
41                         help=f"Annotation provider tier. Available:
{annotation_registry.list_available()}")
42         p.add_argument("--annotations", help="video_ref for the provider (e.g. CSV path for

```

```

the 'file' tier).")
43     p.add_argument("--stage", choices=("candidates", "final"), default="final")
44     p.add_argument("--detector", default=None, help="Restrict candidate scoring to one
detector.")
45     p.add_argument("--iou", type=float, default=0.5)
46     p.add_argument("--tolerance", type=float, default=regression.DEFAULT_TOLERANCE)
47     p.add_argument("--db", default=None)
48     p.add_argument("--baseline", default=regression.DEFAULT_BASELINE_PATH,
help="Baseline JSON path.")
49     p.add_argument("--update-baseline", action="store_true",
50                     help="Snapshot the current numbers as the new baseline instead of
checking.")
51     p.add_argument("--json", dest="json_out", default=None)
52     return p
53
54
55     def main(argv=None) -> int:
56         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
57         args = build_parser().parse_args(argv)
58
59         db_path = args.db or _default_db_path()
60         if not os.path.exists(db_path):
61             print(f"ERROR: database not found at {db_path}. Run the pipeline first.",
file=sys.stderr)
62             return 2
63         db = Database(db_path)
64
65         result = regression.regress_with_provider(
66             db, video_id=args.video_id, video_ref=args.annotations, tier=args.tier,
67             stage=args.stage, iou_threshold=args.iou, detector=args.detector,
68             tolerance=args.tolerance, baseline_path=args.baseline,
69         )
70
71         print(regression.format_result(result))
72
73         if args.update_baseline:
74             regression.save_baseline(result.video_id, result.stage, result.precision,
75                                     result.recall, result.f1, path=args.baseline)
76             print(f"Baseline updated in {args.baseline}.")
77
78         if args.json_out:
79             with open(args.json_out, "w") as f:
80                 json.dump(result.as_dict(), f, indent=2)
81                 print(f"JSON written to {args.json_out}")
82
83         # Non-zero exit on regression so CI can gate on it.
84         return 1 if result.regressed else 0
85
86
87     if __name__ == "__main__":
88         raise SystemExit(main())
89

```

7. user (2026-06-06T15:20:49.075Z)

```

1     import os
2     import sys
3     import argparse
4     import hashlib
5     import json
6     import uvicorn
7     from dotenv import load_dotenv
8
9     load_dotenv()
10    from fastapi import FastAPI, HTTPException

```

```

11     from fastapi.middleware.cors import CORSMiddleware
12     from pydantic import BaseModel
13     from typing import List, Dict, Any, Optional
14
15     from fastapi.staticfiles import StaticFiles
16     from fastapi.responses import HTMLResponse
17     from backend.validators.base import registry
18     from backend.database import Database
19     from backend.indexer.base import segmenter_registry, BasePlaySegmenter
20     from backend.stitcher.compiler import VideoCompiler
21
22     app = FastAPI(title="Sports Highlight Indexer API")
23
24     # Enable CORS for local web interface
25     app.add_middleware(
26         CORSMiddleware,
27         allow_origins=["*"],
28         allow_credentials=True,
29         allow_methods=["*"],
30         allow_headers=["*"],
31     )
32
33     # Serves static frontend files directly
34     os.makedirs("backend/static", exist_ok=True)
35     app.mount("/static", StaticFiles(directory="backend/static"), name="static")
36
37     @app.get("/", response_class=HTMLResponse)
38     def serve_index():
39         index_path = "backend/static/index.html"
40         if os.path.exists(index_path):
41             with open(index_path, "r", encoding="utf-8") as f:
42                 return f.read()
43         return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in  
backend/static/</h3>"
44
45     # Global variables initialized at startup
46     db_instance: Optional[Database] = None
47     global_config: Optional[Dict[str, Any]] = None
48
49     def get_file_md5(filepath: str) -> str:
50         """Calculates file hash to uniquely identify videos and prevent duplicate  
indexes."""
51         hasher = hashlib.md5()
52         with open(filepath, "rb") as f:
53             # Read in chunk sizes of 64kb
54             for chunk in iter(lambda: f.read(65536), b""):
55                 hasher.update(chunk)
56         return hasher.hexdigest()
57
58     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
59         if os.path.exists(config_path):
60             with open(config_path, "r") as f:
61                 return json.load(f)
62         return {}
63
64     # --- API Models ---
65     class ProcessRequest(BaseModel):
66         video_path: str
67         player_pool: Optional[List[str]] = None
68         max_frames: Optional[int] = None
69         segmenter: Optional[str] = None
70
71     class QueryRequest(BaseModel):
72         video_id: str
73         server: Optional[str] = None

```

```

74     receiver: Optional[str] = None
75     duration_min: Optional[float] = None
76     event_type: Optional[str] = None
77
78     class CompileRequest(BaseModel):
79         video_id: str
80         segment_ids: List[int]
81         output_filename: str
82
83     # --- API Endpoints ---
84     @app.get("/api/config")
85     def get_config():
86         return global_config
87
88     @app.post("/api/process")
89     def process_video(req: ProcessRequest):
90         if not os.path.exists(req.video_path):
91             raise HTTPException(status_code=404, detail=f"Video file not found at
92 '{req.video_path}'")
93         video_id = get_file_md5(req.video_path)
94
95         # 1. Run pluggable validators
96         print(f"[API] Running validations for {req.video_path}...")
97         val_results = registry.run_all(req.video_path, global_config)
98
99         # Check if any validator failed
100        failures = [r for r in val_results if not r.passed]
101        if failures:
102            # Save failed processing log
103            db_instance.add_video(video_id, req.video_path, "failed", [r.dict() for r in
104 val_results])
105            return {
106                "video_id": video_id,
107                "status": "failed",
108                "message": "Video failed validation checks.",
109                "results": [r.dict() for r in val_results]
110            }
111
112        # 2. Run segmenter & indexer
113        print(f"[API] Video passed checks. Segmenting and indexing...")
114        db_instance.add_video(video_id, req.video_path, "processed", [r.dict() for r in
115 val_results])
116
117        seg_name = req.segmenter or global_config.get("indexing",
118 {}).get("default_segmenter", "motion")
119        segmenter_cls = segmenter_registry.get(seg_name)
120        if not segmenter_cls:
121            available = list(segmenter_registry.list_available().keys())
122            raise HTTPException(
123                status_code=400,
124                detail=f"Segmenter '{seg_name}' not found. Available segmenters:
125 {available}"
126            )
127
128        print(f"[API] Using segmenter: {seg_name}")
129        segmenter = segmenter_cls(db_instance, global_config)
130        segments = segmenter.process_video(video_id, req.video_path, req.player_pool,
131 req.max_frames)
132
133        return {
134            "video_id": video_id,
135            "status": "success",
136            "message": f"Successfully indexed {len(segments)} play segments using
137 '{seg_name}' segmenter.",

```

```

132         "results": [r.dict() for r in val_results],
133         "play_segments": segments
134     }
135
136 @app.post("/api/query")
137 def query_play_segments(req: QueryRequest):
138     filters = {
139         "server": req.server,
140         "receiver": req.receiver,
141         "duration_min": req.duration_min,
142         "event_type": req.event_type
143     }
144     segments = db_instance.query_play_segments(req.video_id, filters)
145     return {"play_segments": segments}
146
147 @app.post("/api/compile")
148 def compile_highlights(req: CompileRequest):
149     video = db_instance.get_video(req.video_id)
150     if not video:
151         raise HTTPException(status_code=404, detail="Indexed video record not found.")
152
153     # Retrieve matching play segments from db
154     all_segments = db_instance.query_play_segments(req.video_id, {})
155     matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
156
157     if not matched_segments:
158         raise HTTPException(status_code=400, detail="No matching play segments found to
159 stitch.")
160
161     # Extract start/end time pairs
162     time_pairs = [(s["start_time"], s["end_time"]) for s in matched_segments]
163
164     # Run compiler
165     output_dir = global_config.get("output", {}).get("output_dir", "output")
166     compiler = VideoCompiler(output_dir)
167     try:
168         out_path = compiler.compile_highlights(video["filepath"], time_pairs,
169 req.output_filename)
170         return {"status": "success", "filepath": out_path}
171     except Exception as e:
172         raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
173 {str(e)}")
174
175 # --- CLI Debug Utility & Orchestration ---
176 def run_cli_diagnose_clip(video_path: str, timestamp: float):
177     """CLI debugging utility to examine segment properties around a timestamp."""
178     print("=" * 60)
179     print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
180     print("=" * 60)
181
182     cfg = load_config()
183
184     # 1. Validation check diagnostics
185     print("[DIAGNOSTIC] Running validation framework...")
186     results = registry.run_all(video_path, cfg)
187     for r in results:
188         status_symbol = "[PASS]" if r.passed else "[FAIL]"
189         print(f"    {status_symbol} {r.validator_name}: {r.message}")
190         if r.details:
191             print(f"        Details: {r.details}")
192
193     # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
194     print("-" * 60)
195     print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
196     import cv2

```

```

194     cap = cv2.VideoCapture(video_path)
195     if not cap.isOpened():
196         print("[FAIL] OpenCV could not open video.")
197         return
198
199     fps = cap.get(cv2.CAP_PROP_FPS)
200     total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
201
202     start_frame = max(0, int((timestamp - 3.0) * fps))
203     end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
204
205     # Measure frame-by-frame changes in sample region
206     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
207     prev_gray = None
208     motions = []
209
210     for f in range(start_frame, end_frame):
211         ret, frame = cap.read()
212         if not ret:
213             break
214         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
215         gray = cv2.GaussianBlur(gray, (21, 21), 0)
216
217         if prev_gray is not None:
218             diff = cv2.absdiff(prev_gray, gray)
219             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
220             motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
221             motions.append(motion_ratio)
222             prev_gray = gray
223
224     cap.release()
225
226     if motions:
227         avg_motion = sum(motions) / len(motions)
228         threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08)
229         print(f" Sample Frame Count: {len(motions)}")
230         print(f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
231         if avg_motion > threshold:
232             print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
233         else:
234             print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
235     else:
236         print(" [ERROR] No visual frames were extracted in the sample range.")
237
238     print("=" * 60)
239
240     def main():
241         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
242         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
243         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
244         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
245         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
246         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
247         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
248         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N

```

```

sub-sampled frames for debugging")
249
250     available_segmenters = list(segmenter_registry.list_available().keys())
251     parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
252     parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
253     parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
254     parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
255
256     args = parser.parse_args()
257
258     if args.setup:
259         # Invoke setup diagnostics
260         import subprocess
261         print("[INFO] Invoking setup.py...")
262         subprocess.run([sys.executable, "setup.py"])
263         return
264
265     if args.trim_start is not None and args.trim_end is not None:
266         if not args.video_path:
267             print("[ERROR] Please provide --video-path to extract a segment.")
268             return
269
270         # Determine output path
271         out_filename = args.trim_output or "trimmed_segment.mp4"
272         if os.path.isabs(out_filename):
273             out_path = out_filename
274             out_dir = os.path.dirname(out_path)
275             out_name = os.path.basename(out_path)
276         else:
277             out_dir = args.output_dir
278             out_path = os.path.join(out_dir, out_filename)
279             out_name = out_filename
280
281         os.makedirs(out_dir, exist_ok=True)
282         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
283
284         compiler = VideoCompiler(out_dir)
285         try:
286             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
287             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
288         except Exception as e:
289             print(f"[ERROR] Failed to compile trimmed segment: {e}")
290         return
291
292     if args.debug_clip is not None:
293         if not args.video_path:
294             print("[ERROR] Please provide --video-path to debug a specific clip.")
295             return
296         run_cli_diagnose_clip(args.video_path, args.debug_clip)
297         return
298
299     # Initialize Global Config and DB
300     global global_config, db_instance
301     global_config = load_config()
302     # Merge output_dir into config
303     if "output" not in global_config:
304         global_config["output"] = {}
305     global_config["output"]["output_dir"] = args.output_dir
306     os.makedirs(args.output_dir, exist_ok=True)

```

```

307
308     db_path = os.path.join(args.output_dir, global_config.get("output",
309     {}).get("db_name", "sports_indexer.db"))
310     db_instance = Database(db_path)
311
312     # CLI processing mode (no web UI needed)
313     if args.video_path and not args.server:
314         print(f"[CLI] Processing video file: {args.video_path}")
315         if not os.path.exists(args.video_path):
316             print(f"[FAIL] Video file not found: {args.video_path}")
317             return
318
319         video_id = get_file_md5(args.video_path)
320
321         # Validate
322         print("[CLI] Validating...")
323         val_results = registry.run_all(args.video_path, global_config)
324         failures = [r for r in val_results if not r.passed]
325
326         # Save video status
327         status = "failed" if failures else "processed"
328         db_instance.add_video(video_id, args.video_path, status, [r.dict() for r in
329         val_results])
330
331         print("\n" + "="*40)
332         print("VALIDATION SUMMARY")
333         print("="*40)
334         for r in val_results:
335             status_symbol = "[PASS]" if r.passed else "[FAIL]"
336             print(f"{status_symbol} {r.validator_name}: {r.message}")
337         print("="*40 + "\n")
338
339         if failures:
340             print("[FAIL] Video failed checks. Indexing halted.")
341             return
342
343         # Index
344         seg_name = args.segmenter or global_config.get("indexing",
345         {}).get("default_segmenter", "motion")
346         segmenter_cls = segmenter_registry.get(seg_name)
347         if not segmenter_cls:
348             print(f"[FAIL] Segmenter '{seg_name}' not found.")
349             return
350
351         print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
352         segmenter = segmenter_cls(db_instance, global_config)
353         segments = segmenter.process_video(video_id, args.video_path,
354         max_frames=args.max_frames)
355         print(f"[SUCCESS] Process complete. Indexed {len(segments)} play segments inside
356         database: {db_path}")
357         return
358
359         # Start Local Server Mode
360         if args.server or not args.video_path:
361             print(f"[SERVER] Starting local FastAPI server on
362             http://localhost:{args.port}...")
363             uvicorn.run(app, host="127.0.0.1", port=args.port)
364
365     if __name__ == "__main__":
366         main()
367

```

8. user (2026-06-06T15:20:49.943Z)

```
1 {
```

```

2      "sport": "badminton",
3      "ai_provider": "gemini",
4      "ai_model": "gemini-2.5-flash",
5      "validation": {
6          "min_resolution_width": 1280,
7          "min_resolution_height": 720,
8          "min_fps": 29.9,
9          "min_blur_threshold": 20.0,
10         "max_scene_cuts_per_minute": 0.5,
11         "relevance": {
12             "court_green_lower": [
13                 35,
14                 40,
15                 40
16             ],
17             "court_green_upper": [
18                 85,
19                 255,
20                 255
21             ],
22             "court_pixels_min_ratio": 0.05
23         }
24     },
25     "indexing": {
26         "default_segmenter": "gemini",
27         "use_gpu": true,
28         "motion_threshold": 0.08,
29         "min_segment_duration": 3.0,
30         "dead_time_merge_gap": 2.0,
31         "chunk_duration": 90,
32         "chunk_overlap": 10,
33         "detector_model": "fasterrcnn_resnet50_fpn",
34         "detector_score_threshold": 0.5,
35         "yolo_velocity_threshold": 0.15,
36         "yolo_frame_skip": 3,
37         "yolo_tracker": "bytetrack.yaml",
38         "yolo_parallel_chunks": false,
39         "yolo_chunk_workers": 2,
40         "min_action_window_duration": 2.0,
41         "log_yolo_candidates": true,
42         "store_yolo_candidates": true,
43         "ai_handoff_padding": 2.0,
44         "ai_max_workers": 5,
45         "tracknet": {
46             "wsl_distro": "Ubuntu",
47             "repo_dir": "~/models/TrackNetV4",
48             "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
49             "conda_env": "TrackNetV4",
50             "weights_path": "",
51             "queue_length": 5,
52             "stage_in_wsl": true,
53             "wsl_stage_dir": "~/clips",
54             "timeout_sec": 0,
55             "velocity_threshold": 0.02
56         }
57     },
58     "output": {
59         "default_highlights_name": "highlights.mp4",
60         "db_name": "sports_indexer.db"
61     },
62     "stitching": {
63         "begin_padding": 0.5,
64         "end_padding": 0.5,
65         "use_cache": false,
66         "min_shot_count": 1

```

```
67     }
68 }
```

9. user (2026-06-06T15:20:50.024Z)

```
1     import sys
2     import os
3     import subprocess
4     import shutil
5     import json
6
7     def check_command(cmd):
8         """Check if a system command is available."""
9         return shutil.which(cmd) is not None
10
11     def run_pip_install():
12         """Install required Python packages."""
13         print("Installing Python packages from requirements.txt...")
14
15         # Check if NVIDIA GPU is present
16         has_gpu = check_command("nvidia-smi")
17
18         pip_cmd = [sys.executable, "-m", "pip", "install", "--user", "-r",
19 "requirements.txt"]
20
21         if has_gpu:
22             print("[INFO] NVIDIA GPU detected. Configuring PyTorch to download CUDA-enabled
23 wheels.")
24             pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cu124"])
25         else:
26             print("[INFO] No NVIDIA GPU detected. Defaulting to PyTorch CPU wheels.")
27             pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cpu"])
28
29         try:
30             subprocess.check_call(pip_cmd)
31             print("[SUCCESS] Python packages installed.")
32             return True
33         except subprocess.CalledProcessError as e:
34             print(f"[ERROR] Failed to install Python packages: {e}")
35             return False
36
37     def init_default_config():
38         """Initialize a default config.json if it doesn't exist."""
39         config_path = "config.json"
40         if os.path.exists(config_path):
41             print(f"[INFO] {config_path} already exists. Skipping initialization.")
42             return
43
44         default_config = {
45             "sport": "badminton",
46             "ai_provider": "gemini",
47             "ai_model": "gemini-2.5-flash",
48             "validation": {
49                 "min_resolution_width": 1280,
50                 "min_resolution_height": 720,
51                 "min_fps": 29.9,
52                 "min_blur_threshold": 100.0, # Laplacian variance threshold
53                 "max_scene_cuts_per_minute": 0.5 # Continuous raw files should have near 0
54             },
55             "indexing": {
56                 "default_segmenter": "yolo_hybrid",
57                 "use_gpu": True,
58                 "motion_threshold": 0.08,
59                 "min_segment_duration": 3.0,
```

```

58         "dead_time_merge_gap": 2.0,
59         "chunk_duration": 90,
60         "chunk_overlap": 10,
61         "detector_model": "fasterrcnn_resnet50_fpn",
62         "detector_score_threshold": 0.5,
63         "yolo_velocity_threshold": 0.15,
64         "yolo_frame_skip": 3,
65         "yolo_tracker": "bytetrack.yaml",
66         "yolo_parallel_chunks": False,
67         "yolo_chunk_workers": 2,
68         "min_action_window_duration": 2.0,
69         "ai_handoff_padding": 2.0,
70         "ai_max_workers": 5
71     },
72     "output": {
73         "default_highlights_name": "highlights.mp4",
74         "db_name": "sports_indexer.db"
75     },
76     "stitching": {
77         "begin_padding": 0.5,
78         "end_padding": 0.5,
79         "use_cache": False,
80         "min_shot_count": 1
81     }
82 }
83
84 try:
85     with open(config_path, "w") as f:
86         json.dump(default_config, f, indent=4)
87         print("[SUCCESS] config.json initialized with default settings.")
88 except Exception as e:
89     print(f"[ERROR] Failed to create config.json: {e}")
90
91 def run_diagnostics():
92     """Perform system-wide checks for FFmpeg and PyTorch/CUDA."""
93     print("=" * 50)
94     print("SPORTS HIGHLIGHT INDEXER SETUP DIAGNOSTICS")
95     print("=" * 50)
96
97     # 1. Check Python Version
98     print(f"Python Version: {sys.version_info.major}.{sys.version_info.minor}.{sys.version_info.micro}")
99     if sys.version_info.major < 3 or (sys.version_info.major == 3 and
100 sys.version_info.minor < 8):
101         print("[WARNING] Python 3.8+ is highly recommended!")
102     else:
103         print("[PASS] Python version check passed.")
104
105     print("-" * 50)
106
107     # 2. Check FFmpeg / FFprobe
108     ffmpeg_ok = check_command("ffmpeg")
109     ffprobe_ok = check_command("ffprobe")
110
111     if ffmpeg_ok:
112         print("[PASS] FFmpeg is installed and available in PATH.")
113     else:
114         print("[FAIL] FFmpeg is NOT found in your PATH.")
115
116     if ffprobe_ok:
117         print("[PASS] FFprobe is installed and available in PATH.")
118     else:
119         print("[FAIL] FFprobe is NOT found in your PATH.")
120
121     if not (ffmpeg_ok and ffprobe_ok):

```

```

121         print("\n" + "!" * 50)
122         print("CRITICAL: FFmpeg and FFprobe are REQUIRED to run this application.")
123         print("How to install FFmpeg on Windows:")
124         print(" Option 1: Open PowerShell as Administrator and run:")
125         print("         winget install Gyan.FFmpeg")
126         print(" Option 2: Download the build from https://ffmpeg.org/download.html")
127         print("         and add its 'bin' directory to your System PATH environment
variable.")
128         print("!" * 50 + "\n")
129     else:
130         print("[SUCCESS] All essential media command line tools are ready.")
131
132     print("-" * 50)
133
134     # 3. Check ML / GPU (Optional but good)
135     try:
136         import torch
137         cuda_ok = torch.cuda.is_available()
138         if cuda_ok:
139             print(f"[PASS] PyTorch loaded successfully. CUDA (GPU) is AVAILABLE:
{torch.cuda.get_device_name(0)}")
140         else:
141             print("[INFO] PyTorch loaded successfully, but CUDA (GPU) is NOT available.
Falling back to CPU.")
142     except ImportError:
143         print("[INFO] PyTorch is not installed. Relevance checks will run using
lightweight visual heuristics.")
144
145     print("=" * 50)
146     return ffmpeg_ok and ffprobe_ok
147
148 if __name__ == "__main__":
149     init_default_config()
150     media_ok = run_diagnostics()
151     run_pip_install()
152     print("\n[INFO] Setup complete. If diagnostics passed, you can now run the app!")
153

```

10. assistant (2026-06-06T15:20:53.352Z)

I have all the files. Let me resolve the registries referenced across these files to make the edges accurate.

11. user (2026-06-06T15:20:55.376Z)

```

30:class SegmenterRegistry:
43:    def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
46:    def list_available(self) -> Dict[str, str]:
58:segmenter_registry = SegmenterRegistry()

```

12. user (2026-06-06T15:20:56.147Z)

```

backend\annotations\automated_provider.py:24:    annotation_registry,
backend\annotations\automated_provider.py:45:        "annotation_registry.get('auto',
labeler=<fn>, lineage='<model-family>'). "
backend\annotations\automated_provider.py:103:annotation_registry.register(AutomatedProvider)
backend\annotations\file_provider.py:16:    annotation_registry,
backend\annotations\file_provider.py:66:annotation_registry.register(FileProvider)
backend\annotations\base.py:81:class AnnotationProviderRegistry:
backend\annotations\base.py:106:    def list_available(self) -> List[str]:
backend\annotations\base.py:110:annotation_registry = AnnotationProviderRegistry()
backend\annotations\__init__.py:7:from .base import annotation_registry, AnnotationProvider
backend\annotations\__init__.py:11:__all__ = ["annotation_registry", "AnnotationProvider"]

```

13. assistant (2026-06-06T15:21:02.501Z)

I have enough to map these accurately. Let me check the `batch` and `harness`/`regression` signatures quickly to nail the contracts on the edges.

14. user (2026-06-06T15:21:04.660Z)

```
backend\eval\batch.py:22:def resolve_video_path(local_path: Optional[str], videos_root: str) ->
Optional[str]:
backend\eval\batch.py:40:def default_stages_for(segmenter: str, requested: str = "auto") ->
List[str]:
backend\eval\batch.py:61:def aggregate(per_video: List[dict], stages: List[str]) -> Dict[str,
dict]:
backend\eval\batch.py:100:def format_aggregate(agg: Dict[str, dict]) -> str:
backend\eval\harness.py:27:STAGES = ("candidates", "final")
backend\eval\harness.py:66:def evaluate(db: Database, video_id: str, gt_intervals:
List[Interval],
backend\eval\harness.py:92:def report_to_dict(report: EvalReport) -> dict:
backend\eval\harness.py:108:def format_report_text(report: EvalReport, show_failures: bool =
False) -> str:
backend\eval\calibration.py:41:def evaluate(preds: List[Interval], gts: List[Interval],
iou_threshold: float = 0.5) -> Score:
backend\eval\regression.py:30:DEFAULT_BASELINE_PATH = os.path.join("output",
"regression_baseline.json")
backend\eval\regression.py:31:DEFAULT_TOLERANCE = 0.05
backend\eval\regression.py:67:def save_baseline(video_id: str, stage: str, precision: float,
recall: float,
backend\eval\regression.py:128:def regress_with_provider(db: Database, video_id: str, video_ref:
str,
backend\eval\regression.py:150:def format_result(r: RegressionResult) -> str:
```

15. assistant (2026-06-06T15:21:07.673Z)

I have everything needed. Submitting the structured map.

16. user (2026-06-06T15:22:33.137Z)

Structured output provided successfully

17. assistant (2026-06-06T15:22:41.752Z)

Mapped the Orchestration & config subsystem. Structured output submitted covering all seven files.

Key cross-cutting findings worth flagging for the next engineer:

- **Three conflicting `default\_segmenter` defaults**: `config.json` ships `gemini` (costs API calls), `setup.py`'s generated template writes `yolo\_hybrid`, and `backend/main.py`'s code fallback is `motion`. Whichever wins depends on which path you hit.
- **`video\_id` is always MD5-of-whole-file**, used as the join key across DB, eval, and stitching ? but the chunk sizes differ per endpoint (1MB / 8KB / 64KB / 65536). They still agree since all hash the full file. In `run\_phase3\_eval.py` the manifest's `video\_id` is just a human label; the real DB key is the file MD5.
- **Eval CLIs are pure scorers** ? they read predictions from `output/<db\_name>` and never run the pipeline, so they fail with "database not found" / zero predictions if the video wasn't processed first.
- **`backend/main.py` globals** (`db\_instance`, `global\_config`) are only set inside `main()`; importing `app` for an ASGI server without running `main()` leaves them `None` and every endpoint NPEs. uvicorn host is hardcoded to `127.0.0.1`.
- **`setup.py`'s generated config diverges** from the checked-in `config.json` (missing `validation.relevance` and `indexing.tracknet` blocks, different blur threshold/segmenter) ? but it no-ops if `config.json` already exists.